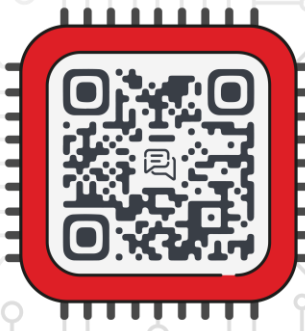




2

10

1200 sp

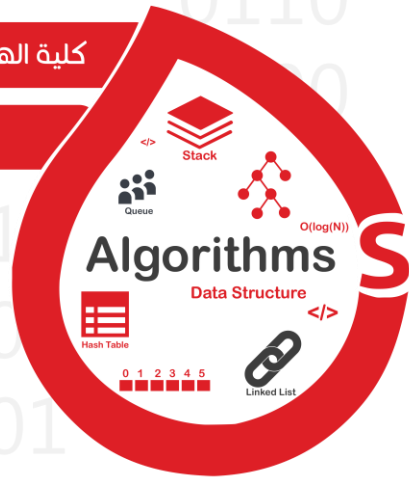
نظري

كلية الهندسة المعلوماتية

السنة الثانية

التعقيدين الذاكريّ و الزمّنيّ

د. أبي صندوق



RB Informatics; 19/10/2025

خوارزميات 1

لكل مسألة عدّة حلول مختلفة، وكل حل نصل إليه عن طريق خوارزمية معينة، و لكن كيف نحدّد أفضل خوارزمية لحل المسألة؟؟؟

هنا يظهر دور التعقيد الزمني.. فما هو التعقيد ؟

للتعقيد نوعان:

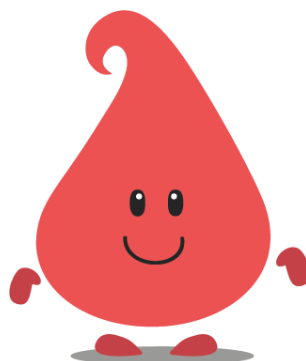
- تعقيد زمني
- تعقيد ذاكري

و لكن أهمّ معيار للحكم على الخوارزمية هو التعقيد **الزمني**... لماذا؟؟؟

- التعقيد **الذاكري** يمكن التحكم به، حيث أنّ الذاكرة توفّرت بشكل كبير في عصرنا الحالي فلا تشكّل مشكلة كبيرة لنا.
- أمّا **الزمن** فهو محدود و لا يمكن التّحكّم به.

التّعقيد الذاكريّ

نعلم أنّ المتغيّر ذو النوع "int" يحجز في الذاكرة 4 bytes و بالتالي إذا كان لدينا مصفوفة من النوع "int" تحوي n عنصراً، فإنّ المصفوفة تحجز في الذاكرة n × 4 bytes، و لحساب التعقيد الذاكريّ كاملاً نقوم بجمع كل الأحجام التي حجزتها المتغيّرات و بنى المعطيات المستخدمة في برنامجنا.



تذكرة أريسيزية:

Size (bytes)	Data type
4	float
8	double
1	char
1	boolean

Size (bytes)	Data type
1	byte
2	short
4	int
8	long long



التعقيد الزمني

التعقيد الزمني دالة رياضية تصف مقدار الزمن الذي تستهلكه الخوارزمية لحل المسألة كنسبة إلى حجم المدخلات n ، أي نقيس الزمن بعدد العمليات التي تجري ضمن الكود [ليس بالثواني الفعلية؛ لأن الثواني تختلف باختلاف الأجهزة و مواردها]. $\Leftarrow$ (التعقيد الزمني = زمن تنفيذ العمليات).

ملاحظة:

التعقيد الزمني مهم للمقارنة بين الخوارزميات من حيث كفاءتها خصوصاً عندما يكبر حجم المدخلات.

كيف نحسب التعقيد الزمني؟؟

لحساب التعقيد الزمني نقوم بالخطوات التالية:

1. نحدد العملية الأساسية (التي تتكرر أكثر).
2. نكتب المعادلة.
3. نبسط المعادلة بحذف الثوابت.

و لتوضيح الفكرة سنعرض بعض الأمثلة:

مثال 1 (حساب مجموع عناصر مصفوفة من الحجم 5):

```

1 public class SumA {
2     public static void main(String[] args) {
3         int a[] = {1, 2, 3, 4, 5};
4         int n = a.length;
5         int sum = 0;
6         for (int i = 0; i < n; i++) {
7             sum += a[i];
8         }
9         System.out.println(sum);
10    }
11 }
```

■ زمن التنفيذ اللازم للكود هو مجموع زمن تنفيذ كل عملية.

السطر	العمليات	Time	Cost
3	تهيئة مصفوفة بطول 5	1	1
4	تهيئة متغير	1	1
	إسناد كل عنصر في المصفوفة (في مثالنا 5 و لكن من أجل n عنصر)	n	1
5	تهيئة المجموع	1	1
6	حلقة for تهيئة i : تتم مرة واحدة	1	1
	فحص الشرط	n+1	1
	عملية الجمع (i++)	n	1
7	عملية الجمع	n	1
9	الطباعة	1	1

$$T(n) \rightarrow \Sigma(cost \times Time)$$

$$T(n) = (1)(1) + (1)(1) + (1)(n) + (1)(1) + (1)(1) + (1)(n+1) + (1)(n) + (1)(1)$$

$$\Rightarrow T(n) = 4n + 6$$

نحذف الثوابت فيكون التعقيد خطي $O(n)$.

هذه المعادلة تمثل التركيب الجبري [عدد العمليات اللازم لتنفيذ الكود]، و لتحويلها للتابع الزمني نضرب بزمن العملية.

ملاحظة:

التركيب الجبري واحدته "خطوة" أو "عملية" (instruction)، نحوله إلى تابع زمني عن طريق ضربه بزمن تنفيذ كل عملية. مثلاً $10^{-8} * () \leftarrow$ و تصبح واحدته ثانية.

مثال 2:

لدينا مصفوفة علامات، و نريد معرفة كم طالب حصل على 80.

1. طريقة العدّاد (حيث إنّ $key = 80$):

```

1 int find (int []A, int key) {
2     int counter = 0;
3     for (int i = 0; i < A.length; i++) {
4         if (A[i] == key) {
5             counter ++;
6         }
7     }
8     return counter;
9 }

```

رقم السطر	Time	Cost
2	1	1
3	n	2
4	n	1
5	$0 \leq x \leq n$	1
8	1	1

في السطر 2: تم تهيئة عداد و إسناد قيمة له.

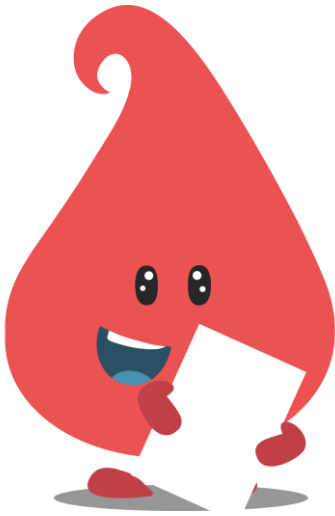
في السطر 3: عملية الجمع و المقارنة و تهيئة المتغير i.

تساؤل: لماذا الزمن n ؟؟**الإجابة:** بفرض عدد عناصر المصفوفة n عنصر؛ فإنّ الحلقة ستتكرّر n مرّة.و لكن الشرط في الحقيقة $i < A.length$ سيتكرّر $n+1$ مرة، لأنه:في آخر مرة عندما $i = n$ (أي انتهت عناصر المصفوفة) هنا سيتمّ محض الشرط و انتهاء الحلقة لأنّ الشرط اختلف.و بما أنّ $n + 1$ لن تؤثر على التعقيد لأنها ثابتة؛ فليس من الضروري وضعها تحت عنوان Time.في السطر 4: المقارنة ستتّم عند كلّ عنصر في المصفوفة؛ أي n مرّة.في السطر 5: لماذا $0 \leq x \leq n$ ؟؟؟

الشرط ممكن أن يتحقق أو لا.

 $0 \leq x$: من الممكن ألا نجد أي علامة 80 ← $counter = 0$. $x \leq n$: من الممكن أن تكون كلّ العلامات 80.

وهنا نأتي لمناقشة ثلاث حالات:



1. أسوأ حالة $x = n$
2. الحالة الوسطى $x = \frac{n}{2}$
3. أفضل حالة $x = 0$

لنكتب المعادلة أو التركيب الجبري في الحالة العامة:

$$\Sigma \text{ operations} = 3n + x + 2 = T(n, x)$$

تابع بالنسبة لـ x و n .

$$T_{Best}(n) = 3n + 2 \rightarrow x = 0$$

$$T_{Worst}(n) = 4n + 2 \rightarrow x = n$$

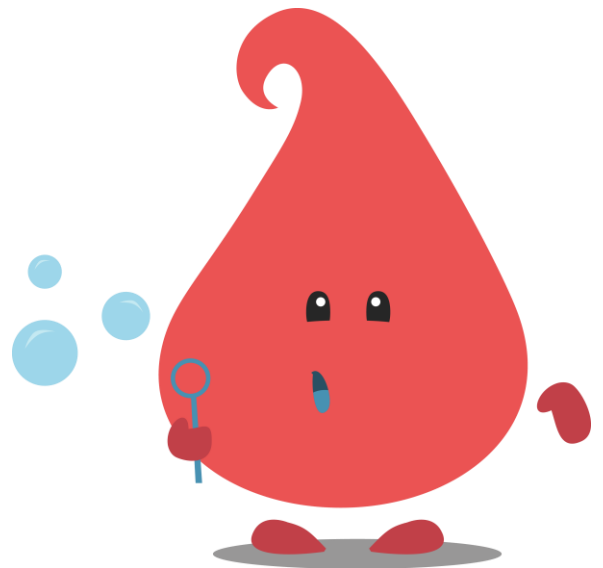
تعقيده الزمني "order" $\leftarrow O(n)$ خطّي.

ملاحظة هامة: في الكود السابق، عدد العمليات اللازمة لتنفيذ الخوارزمية يرتبط بعاملين x, n ؛ أي عدد العمليات مرتبط بحجم المصفوفة؛ فكلما زاد عدد العناصر (n أي) زاد عدد مرات تنفيذ العمليات $x \leftarrow$ متعلق بزيادة n طرداً.

2. الطريقة الثانية:

1. ترتيب المصفوفة
 2. نبحث عن أصغر موضع (index) قيمته 80 و أكبر موضع قيمته 80.
 3. عدد الدرجات التي تساوي 80 هو:
- عدد العناصر = (آخر موضع - أول موضع) + 1

```
1 int find2 (int []A, int key) {
2     Arrays.sort(A);
3     int n = A.length;
4     int first_index = -1;
5     int last_index = -1;
6     // البحث عن أول موضع قيمة رقمية
7     for (int i = 0; i < n; i++) {
8         if (A[i] == 80) {
9             first_index = i;
10            break;
11        }
12    }
13    // البحث عن آخر موضع
14    for (int i = n-1; i >= 0; i--) {
15        if (A[i] == 80) {
16            last_index = i;
17            break;
18        }
19    }
```



```

20 //حساب عدد الدرجات
21 int counter = 0;
22 if (first_index != -1 && last_index != -1) {
23     counter = last_index - first_index + 1;
24 }
25 return counter;
26 }

```

السطر	Cost	Time
25-23-22-21-17-16-10-9-5-4-3	1	1
14 - 7	1	$0 < x \leq n$
2	$n \log n$	1

ملاحظة: عند السطر 2

إن التابع `Array.sort(A)` هو عبارة عن خوارزمية فرز تعتمد على `Time sort` في Java فقط. و تعقيدها $n \log n$.

[استخدام هذا التابع للترتيب يحتاج Arrays في بداية البرنامج] ← `import java.util.Arrays`.

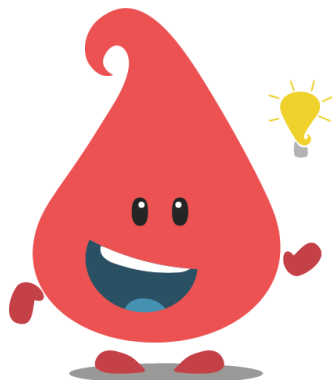
تعقيد التمرين (الطريقة الثانية):

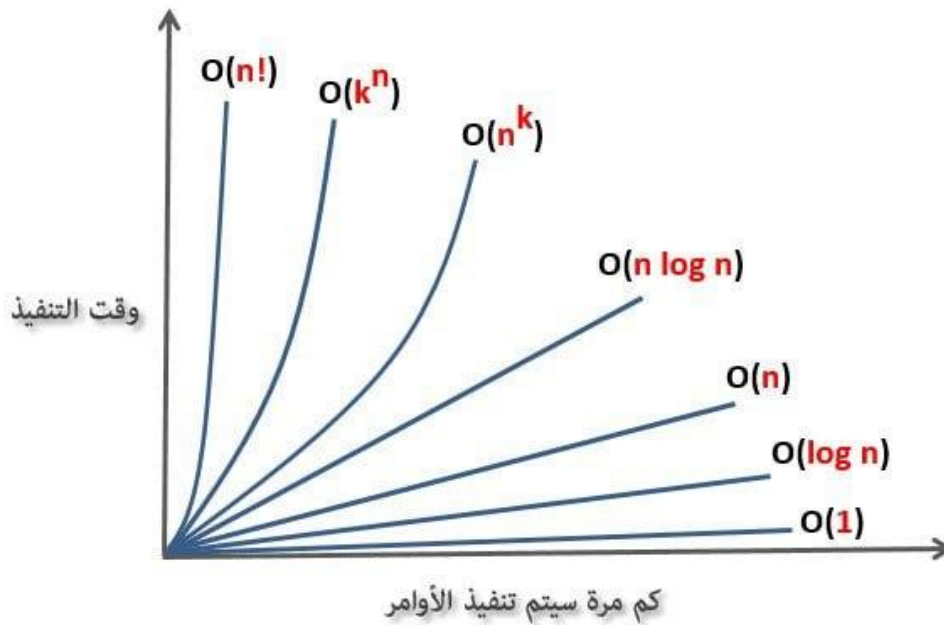
بعد الجمع و إهمال الثوابت $T(n) = n \log n + 6n \leftarrow O(n \log n)$.
حسب مخطط التسارع فإن $n \log n > n$.

و الآن.. أي طريقة هي الأفضل ؟ الأولى أم الثانية ؟؟

- تعقيد الأولى $\leftarrow O(n)$.
- تعقيد الثانية $\leftarrow O(n \log n)$.
- و ذكرنا سابقاً أن $\leftarrow n \log n > n$.

إذاً الخوارزمية الأولى ستأخذ زمن أقل، أي أنها أسرع و أفضل للحل.





$O(1)$: الثوابت تسارعها ثابت دوماً و صغير جداً (constant).

$O(\log n)$: التسارع اللوغاريتمي (logarithmic).

$O(n)$: التسارع الخطي (linear).

$O(n \log n)$: (log-linear).

$O(n^k)$: التابع المرفوع لقوة k .

$O(k^n)$: (exponential).

$O(n!)$: العامل ← أسوأ تعقيد شهده التاريخ X_X.

$$O(1) \leq O(\log n) \leq O(n) \leq O(n \log n) \leq O(n^k) \leq O(k^n) \leq O(n!)$$

ملاحظة:

كل تعقيد يعبر عن تصرف التابع في اللانهاية، فإذا كانت $a_0 + a_1 n + \dots + a_k n^k = T(n)$

فإن $O(n^k) = T(n)$ ، لأن n^k هو الحد المهيمن عندما $n \rightarrow \infty$.

لنفرض لدينا تابع ما فعند استدعائه في الـ main يكون استدعاؤه عبارة عن تعليمة واحدة و لكن لا ننسى أن نعدّ التعليمات بداخل التابع أيضاً.

التعقيد الزمني للأمثلة في المحاضرة السابقة

المثال الأول:

1. الطريقة الأولى: $O(1)$

نحتاج لثلاث عمليات تقسيم و هي عمليات بسيطة، كل عملية $O(1)$ و عملية الطباعة 1.
عملية المقارنة ستتم ضمن تعليمة if و هي تعليمة دون حلقة for تكرر 3 مرات للوصول عدد، و تعقيدها $O(1)$.
فتعقيد الطريقة أو الخوارزمية الأولى ثابت نعبر عنه بـ $O(1)$ مهما كان مجموع الثوابت.

```
1 int lentils=5000 , bulgur = 5000, oil=1000;
2 int lenNedded=100, bulNeeded=200, oilNedded=20;
3
4 int lenMeals= lentils/lenNedded;
5 int bulMeals= bulgur/bulNeeded;
6 int oilMeals=oil/oilNedded;
7
8 int ans=min(lenMeals,min(bulMeals,oilMeals));
9 system.out.println(ans);
```

2. الطريقة الثانية:

التعقيد سيكون من عدد الوجبات النهائية.

لنفرض n وجبة $\leftarrow O(n)$ أي العداد نفسه... لماذا؟؟

نستمر بالجمع حتى نصل لأحد الأوزان، أي انتهت كميتها؛ و هنا سيكون العداد ازداد بعدد مرات الجمع و نريد طباعته
فعدد مرات تكرار العمليات هو نفسه العداد.

```
1 int lentils=5000 , bulgur = 5000, oil=1000;
2 int lenNedded=100, bulNeeded=200, oilNedded=20;
3
4 int lenUsed= 0;
5 int bulUsed= 0;
6 int oilUsed=0;
7 int counter=0;
8
9 while ( lenUsed<lentils && bulUsed < bulgur && oilUsed < oil )
10 {
11     lenUsed+=lenNedded;
12     bulUsed+=bulNeeded;
13     oilUsed+=oilNedded;
14     counter++;
15 }
16 system.out.println(counter);
```


3. الطريقة الثالثة:

 n عدد المنتجات النهائي .. $O(n)$.

```

1  int lentils=5000 , bulgur = 5000, oil=1000;
2  int lenNedded=100, bulNeeded=200, oilNedded=20;
3
4  int lenUsed= 0;
5  int bulUsed= 0;
6  int oilUsed=0;
7  int mul=1;
8
9  while ( lenUsed<lentils  && bulUsed < bulgur && oilUsed < oil )
10     {
11         lenUsed = lenNedded*mul;
12         bulUsed = bulNeeded*mul;
13         oilUsed = oilNedded*mul;
14         mul++;
15     }
16
17  system.out.println(mul-1);

```

4. الطريقة الرَّابِعة:

التعقيد هو $\log$ العنصر الكبير.لنفترض أنه أدخل $x > n \leftarrow x \leftarrow O(\log x)$.

سنتعلم فيما بعد كيفية تطبيق خوارزمية البحث الثنائي (binary search).

5. الطريقة الخامسة:

ليست خوارزمية؛ لأنها غير منتهية، فتعقيدها الزماني سيستمر مدى الحياة $O(x \times m)$.حيث أن: x : عدد المحاولات. m : عدد الوجبات.و لكن x قد تصل لأرقام هائلة و هذا سيء و يسبب مشاكل.

المثال الثاني:

1. الطريقة الأولى:

في أسوأ الحالات (و هي عندما تكون المصفوفة مرتبة تنازلياً) يكون التعقيد $O(n^2)$ ؛ بسبب وجود حلقتين متداخلتين.

أما أفضل الحالات (عندما تكون المصفوفة مرتبة) فيكون التعقيد $O(n)$

و لكن.. أي التعقيد أدق و أعمّ ؟؟

$O(n^2)$ أدق و أعمّ... لأن n مهمة أمام n^2 ، و دائماً ننظر لتعقيد أسوأ حالة. $(n^2 > n)$

2. الطريقة الثانية:

التعقيد هو $O(n^2)$ ؛ بسبب وجود حلقتين متداخلتين كل منهما تسير على n عنصر $\leftarrow n \times n$.

3. الطريقة الثالثة:

نحتاج لتكرار المصفوفة أي السير عليها عدة مرات حتى ضمان الترتيب الكامل و الصحيح لها.
فبتبديل i ب $i+1$ فقط لن يكفي.

ملاحظات على الهامش:

1. كيف نبرهن أن الخوارزمية حل للمسألة ؟

بالتجريب ؟؟... أكيد لاء -_-

لا يمكن أن نبرهن أن الخوارزمية حل للمسألة بالتجريب، بل نلجأ للبرهنة بالاستقراء..

من أجل n هل الخوارزمية صحيحة من أجل $n + 1$ ؟؟؟

بافتراض أن المسألة تم حلها إلى i من العناصر؛ إذاً نبرهن انها محلولة من أجل $i+1$ أيضاً.

2. لا يمكن تحديد كفاءة خوارزمية بالنظر إليها وحدها؛ بل بالمقارنة بينها و بين خوارزمية أخرى لنفس المسألة.



دمتم سالمين